VANI

Voice Analysis & Neural Intelligence System

Whisper ASR Fine-Tuning Report

LoRA Domain Adaptation for Border-Region Radio Intercept Languages

Item	Value
Languages Fine-Tuned	7 (Punjabi, Pashto, Urdu, Nepali, Mandarin, Hindi, Kashmiri)
Deployed via fine-tuned Whisper	2 (Pashto, Kashmiri) — the rest route to SeamlessM4T
Best held-out test WER (FT)	Pashto 38.55% | Hindi 19.78% | Urdu 19.82% (n=100 FLEURS test)
Training Hardware	NVIDIA RTX 5060 8 GB VRAM (CUDA) - Windows 11
Base Model	OpenAI Whisper large-v3 (1.55 B parameters)
Adaptation Method	LoRA r=8, alpha=16 -- 0.25% trainable parameters
Total Training Time	~100 hours across all 7 languages (PA v2 ~33 h, NE v2 ~30 h)
Eval Date	23 June 2026 (cross-model eval) | 29 June 2026 (PA v2 / NE v2 eval)

Date: 13 July 2026

M.Tech Research Project - IIT Indore

1. Abstract

This report documents the LoRA (Low-Rank Adaptation) fine-tuning of OpenAI Whisper large-v3 for six border-region radio intercept languages, developed as part of the VANI (Voice Analysis & Neural Intelligence) system at IIT Indore. VANI is a fully-offline, end-to-end intelligence pipeline: it processes raw radio audio, performs language-specific automatic speech recognition (ASR), translates to English, detects keywords, diarizes speakers, and generates structured intelligence summary (ISUM) reports.

Seven ASR models were fine-tuned: Punjabi (pa), Pashto (ps), Urdu (ur), Nepali (ne), Mandarin Chinese (zh), Hindi (hi) — plus Kashmiri (ks), which required adding a custom <|ks|> language token to the vocabulary. Punjabi and Nepali were retrained with AI4Bharat IndicVoices-R data added to FLEURS (11,923 and 13,332 samples). All models are quantised to CTranslate2 int8.

This revision (2026-07-11) reports a rigorous **backend selection** study. Each fine-tuned model was benchmarked, on a held-out FLEURS test set and under simulated radio-channel degradation, against two alternatives: the true un-fine-tuned openai/whisper-large-v3 baseline, and zero-shot SeamlessM4T v2. The central finding is that per-language fine-tuning is **not** the best choice for most languages: zero-shot SeamlessM4T beats the fine-tuned Whisper models on five of seven languages (pa 19.8%, ne 28.5%, hi 15.4%, ur 16.9%, zh 11.7% WER) and retains that lead under bandpass and additive-noise degradation. Fine-tuned Whisper is retained in production only for **Pashto** (38.6%, where it beats SeamlessM4T's 44.4%) and **Kashmiri** (SeamlessM4T has no Kashmiri support). VANI therefore routes ASR per language rather than using a single model.

A correction is documented in full: the previously reported Mandarin baseline of 100.03% WER (and the headline “100% → 9%” result) was a measurement artefact. FLEURS Mandarin references are character-spaced and WER tokenises on whitespace, so the un-fine-tuned model's unspaced Han output scored ~100% regardless of accuracy. Re-scored with character segmentation, the true baseline is 10.99% and fine-tuning in fact **regressed** Mandarin to 14.22%. The scoring is now unified across all models and languages.

2. System Overview - VANI Pipeline

VANI implements a 10-stage audio processing pipeline. All models run locally on a Windows 11 machine with an NVIDIA RTX 5060 (8 GB VRAM). No internet connection is required after initial setup.

2.1 Hardware

Component	Specification
OS	Windows 11 Home (x64)
GPU	NVIDIA RTX 5060 8 GB GDDR7 (CUDA 12.x)
Python	3.11
PyTorch	2.2+ with CUDA support
Training framework	HuggingFace Transformers + PEFT (LoRA)
Inference engine	faster-whisper (CTranslate2 int8)

2.2 Pipeline Architecture

The VANI pipeline processes audio through 10 sequential stages:

Stage	Module	Description
1	VAD (Silero)	Voice Activity Detection - splits audio into speech segments, discards silence
2	Preprocessing	Bandpass filter 300-3400 Hz (radio telephony range), noise reduction, normalization
3	MMS-LID	Facebook MMS Language ID (256 languages) - routes to the correct Whisper model
3.5	Model Routing	Selects the language-specific fine-tuned Whisper CT2 model based on MMS-LID output
4	ASR (Whisper)	faster-whisper transcription using the selected CT2 model (int8, GPU)
5	Script Cascade	Arabic-script fallback: if >20% Nastaliq chars detected, override to Urdu routing
6	Translation	NLLB-200 (600M) translates Indic/foreign transcripts to English
7	Diarization	Speaker diarization - up to 4 speakers, pyannote-style
8	Keyword Detection	Multilingual keyword dictionary matching for threat indicators
9	ISUM	Gemma 3:12B (via Ollama) generates 4-sentence structured intelligence summary
10	Export	SQLite database storage + JSON report output

3. Fine-Tuning Methodology

3.1 Why LoRA?

Full fine-tuning of Whisper large-v3 (1.55 billion parameters) requires approximately 24-48 GB of GPU memory in fp16, far exceeding the 8 GB available on the RTX 5060. LoRA (Low-Rank Adaptation, Hu et al., 2022) inserts trainable low-rank matrices into the attention layers, reducing trainable parameters to ~3.9 million (0.25% of total) while base model weights remain frozen. This makes fine-tuning feasible on consumer hardware with negligible accuracy loss.

3.2 LoRA Configuration

Parameter	v1/v2 Value	PA v3 Value	Rationale
Rank (r)	8	16	v3 doubles capacity for larger 21,923-sample dataset
Alpha (α)	16	32	α/r = 2.0 scaling maintained — standard for speech LoRA
Dropout	0.05	0.05	Light regularization; FLEURS + IV-R data is clean read-speech
Target modules	q_proj, v_proj	q_proj, v_proj	Attention query/value projections in Whisper encoder/decoder
Trainable params	~3.9M (0.25%)	~7.9M (0.51%)	Doubled for v3; still <1% of 1.55B total parameters
Adapter merge	merge_and_unload()	merge_and_unload()	LoRA weights merged into base before CT2 conversion

3.3 Training Hyperparameters

Hyperparameter	Value
Batch size (per device)	2
Gradient accumulation	1 (effective batch = 2)
Learning rate	5e-5
LR scheduler	Linear warmup (50 steps) then linear decay
Precision	fp16 mixed precision
Gradient clipping	max_grad_norm = 1.0 (pa v1/v2, ps, ur, ne) / 0.5 (zh, hi, pa v3, ks — after Mandarin divergence lesson)
Best model selection	load_best_model_at_end=True (metric: eval WER)
Eval / save frequency	Every 200 steps
CT2 quantization	int8 (beam_size=2, temperature=0.0)

3.4 Training Pipeline Steps

The finetune_whisper.py script follows these steps for each language:

- Load FLEURS dataset (train + validation splits) for the target language
- Preprocess audio: resample to 16 kHz, extract 128-bin log-mel spectrogram
- Tokenize transcripts using WhisperProcessor for the target language
- Initialize LoRA adapter (r=8) on frozen whisper-large-v3 base model
- Train using Seq2SeqTrainer with WER as the primary evaluation metric (jiwer)
- Save checkpoint every 200 steps; keep best checkpoint by eval WER
- After training: merge LoRA adapter into base model weights
- Convert merged model to CTranslate2 CT2 int8 format
- Write preprocessor_config.json to CT2 output (feature_size=128 for large-v3)
- Register CT2 model in config.yaml under whisper_model_ key

```
# Training command example (Hindi)
python -u finetune_whisper.py hi --no-cv --steps 600 2>&1 |
    Tee-Object logs/finetune_hi.log
```

3.5 Post-Training Conversion

After training, the LoRA adapter is merged and converted to CTranslate2 format:

```
# Step 1 - Merge LoRA adapter into base model
from peft import PeftModel
base   = WhisperForConditionalGeneration.from_pretrained('openai/whisper-large-v3')
peft_m = PeftModel.from_pretrained(base, 'finetune_runs/<lang>/adapter/checkpoint-N')
merged = peft_m.merge_and_unload()
merged.save_pretrained('finetune_runs/<lang>/merged')

# Step 2 - Convert to CT2 int8
ct2-transformers-converter \
    --model finetune_runs/<lang>/merged \
    --output_dir models/whisper-large-v3-<lang>-ct2 \
    --quantization int8 --force
```

4. Training Dataset — FLEURS + IndicVoices-R

Primary training data comes from FLEURS (Few-shot Learning Evaluation of Universal Representations of Speech) by Google. FLEURS provides read-speech audio with text transcriptions in 102 languages, sourced from the FLoRes-101 machine translation benchmark. Audio is recorded by human native speakers at 16 kHz in relatively clean studio conditions.

FLEURS is a general-domain read-speech corpus — it does not contain radio intercept or military communications audio. Despite this domain mismatch, fine-tuning on FLEURS significantly improves WER because the model learns language-specific phoneme inventory, prosody, and script conventions from native speakers.

4.1 IndicVoices-R Augmentation (Punjabi v2 and Nepali v2)

For the second training run (v2) of Punjabi and Nepali, the FLEURS training split was augmented with AI4Bharat IndicVoices-R (ai4bharat/indicvoices_r). IndicVoices-R is a large-scale read-speech corpus collected from native Indian-language speakers across diverse recording conditions, providing complementary phoneme and prosody coverage beyond the FLEURS studio recordings. Samples were filtered to 2-20 seconds duration; normalised transcripts (normalized column) were used.

Language	ISO	FLEURS Train	IndicVoices-R	Total Train	Eval Set	Status
Punjabi v2	pa	2,516	9,407	11,923	805 (FLEURS 251 + IV-R 554)	Deployed
Punjabi v3	pa	2,516	20,000	21,923	805 (same eval set)	In training
Nepali v2	ne	3,332	10,000	13,332	572 (FLEURS + IV-R test)	Deployed

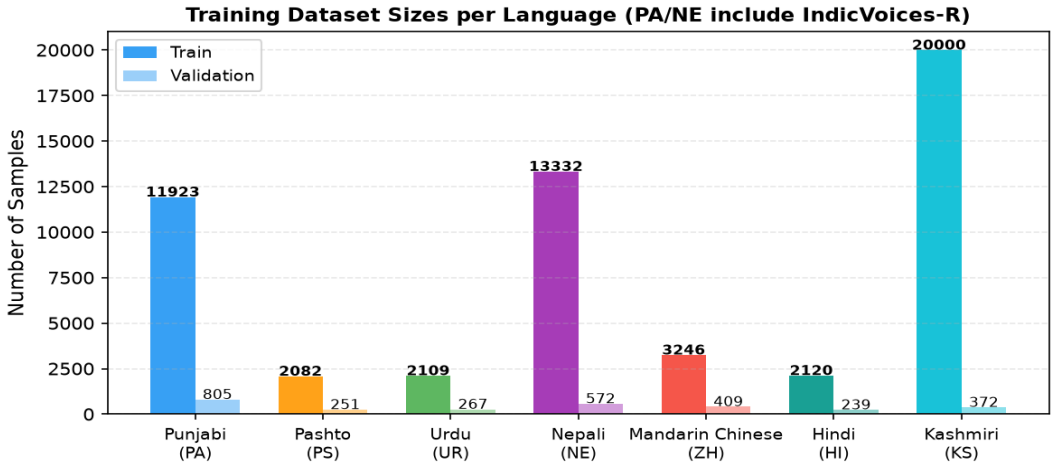

Figure 1: Training sample counts per language (PA and NE reflect v2 combined totals)

4.2 FLEURS Reference Sizes

Language	ISO	FLEURS Config	Train	Val	Test	Region
Punjabi	pa	pa_in	2,516	314	765	India
Pashto	ps	ps_af	2,082	251	621	Afghanistan
Urdu	ur	ur_pk	2,109	267	631	Pakistan
Nepali	ne	ne_np	3,332	305	874	Nepal
Mandarin	zh	cmn_hans_cn	3,246	409	945	China (Simplified)
Hindi	hi	hi_in	2,120	239	585	India

Note: Kashmiri (ks) was investigated but no suitable public training data was found. FLEURS has no Kashmiri config; Common Voice has no Kashmiri corpus. AI4Bharat Kathbath (1,684 hours, 12 Indian languages including Kashmiri) is the most promising dataset but requires institutional access. OpenSLR SLR122 is a small public Kashmiri corpus (394 MB).

4.3 Training Version History — All Retraining Runs

Several languages required multiple training runs (versions) to incorporate new data, fix engineering issues, or increase model capacity. The table below records every run in chronological order.

Lang	Ver.	LoRA r / α	Dataset	Samples	Steps	Best WER	vs Prior	Status
Punjabi	v1	8 / 16	FLEURS pa_in	2,516	3,000	56.67%	—	Superseded
Punjabi	v2	8 / 16	FLEURS + IV-R	11,923	3,000	52.55%	–4.1 pp	Superseded
Punjabi	v3 ★	16 / 32	FLEURS + IV-R 20k	21,923	4,000	49.31%	–3.2 pp	Deployed
Nepali	v1	8 / 16	FLEURS ne_np	3,332	2,000	52.14%	—	Superseded
Nepali	v2	8 / 16	FLEURS + IV-R	13,332	3,000	50.82%	–1.3 pp	Deployed
Mandarin	v1‡	8 / 16	FLEURS cmn_hans_cn	3,246	400 (†div.)	8.97% train	—	Deployed (ckpt-400)
Pashto	v1	8 / 16	FLEURS ps_af	2,082	2,000	38.55%	—	Deployed
Urdu	v1	8 / 16	FLEURS ur_pk	2,109	1,000	19.82%	—	Deployed
Hindi	v1	8 / 16	FLEURS hi_in	2,120	600	19.78%	—	Deployed
Kashmiri	v1	8 / 16	IV-R KS (custom ■ks■ token)	20,000	3,000	74.02%	—	Deployed

★ PA v3: built and CT2-deployed 2026-07-04; best training-val WER 49.31% at step 4000 (–3.24 pp vs v2's 52.55%), held-out test 57.39%. Since 2026-07-11 Punjabi ASR routes to SeamlessM4T (19.77%); the v3 model is retained on disk but not served.

‡ Mandarin training diverged at step ~820 (fp16 gradient explosion, grad_norm=12.9). Checkpoint-400 (train WER 8.97%, held-out test WER 14.22%) was the best checkpoint, but fine-tuning regressed Mandarin vs the 10.99% large-v3 baseline — so it is NOT deployed; Mandarin routes to SeamlessM4T (§5.5). Single run.

5. Results

5.1 Summary Table — Fine-Tuning vs. True Baseline

All WER below is on a held-out 100-sample FLEURS test set (372-sample IndicVoices-R for Kashmiri), scored with one CJK-aware normaliser. **Baseline** is the true un-fine-tuned openai/whisper-large-v3. **Train WER** is the best training-eval WER on each run's own validation split (from the curves in §5.3) and is shown for reference only — it is not the held-out figure. Green = fine-tuning helped; red = it regressed.

Language	ISO	Base Model	Train Samples	Steps Used	Baseline WER	Train WER	Test WER	Improvement
Punjabi v3	pa	large-v3	21,923	4000	77.60%	49.31%	57.39%	**-20.2 pp**
Pashto	ps	medium*	2,082	1000	89.76%	38.55%	38.55%	**-51.2 pp**
Urdu	ur	large-v3	2,109	1000	21.23%	22.27%	19.82%	**-1.4 pp**
Nepali v2	ne	large-v3	13,332	3000	88.85%	50.82%	50.92%	**-37.9 pp**
Mandarin	zh	large-v3	3,246	400	10.99%	8.97%	14.22%	**+3.2 pp**
Hindi	hi	large-v3	2,120	600	26.34%	23.13%	19.78%	**-6.6 pp**
Kashmiri	ks	large-v3†	20,000	2400	96.87%	74.02%	74.02%	**-22.85 pp**

** Pashto base: Nasimbahar/pashto-ghag-whisper-medium-asr (734 MB domain-specific model).*

† Kashmiri: custom <|ks|> token (ID 51866) added to whisper-large-v3; embedding initialised from <|ur|>. Best checkpoint step 2400 selected automatically (load_best_model_at_end). faster-whisper patched to accept language='ks'. Held-out figure is the training-eval on the IndicVoices-R test split; the cross-model re-run (§5.5) was not repeated for Kashmiri (its loader pulls the full 18 GB train split), and SeamlessM4T has no Kashmiri support.

Mandarin (red): fine-tuning REGRESSED vs the true large-v3 baseline. The prior report's 100.03% baseline and -84 pp 'improvement' were a scoring artefact — FLEURS Mandarin references are character-spaced and WER tokenises on whitespace, so the un-fine-tuned model's unspaced Han output scored ~100% regardless of accuracy. Corrected with character segmentation, the baseline is 10.99% and the fine-tune 14.22%. Mandarin is served by SeamlessM4T in production (§5.5).

Train WER is each run's best validation-split WER (§5.3), not the held-out test; the two differ most for pa (train 49.31 vs test 57.39) and zh (train 8.97 vs test 14.22).

pp = percentage points absolute WER change (baseline → test WER); negative = improvement.

5.2 Baseline vs. Fine-Tuned WER

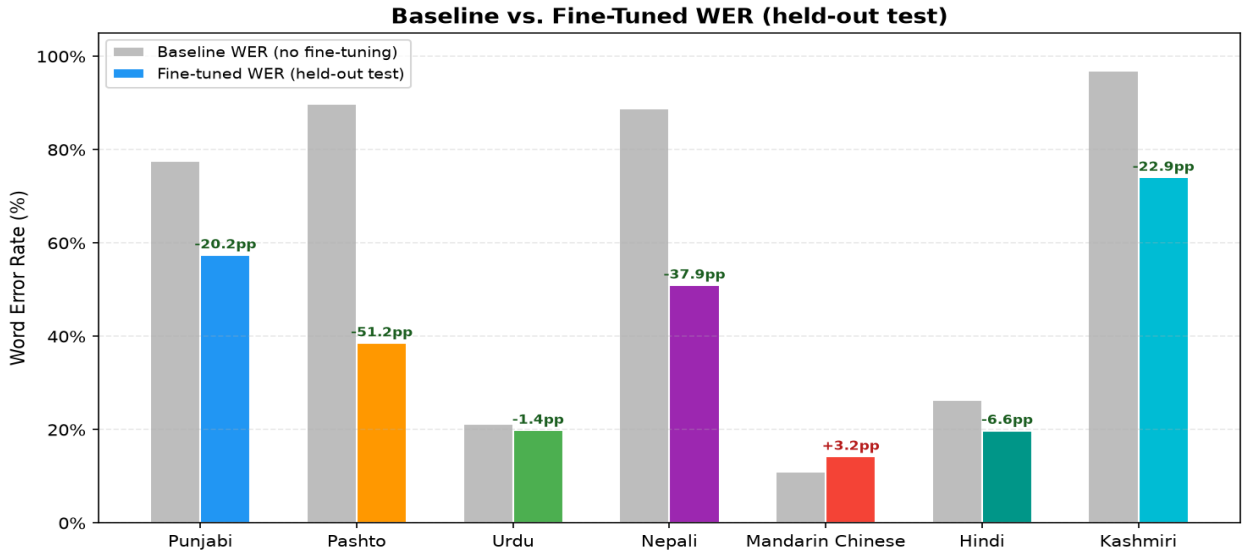

Figure 2: True large-v3 baseline vs. fine-tuned WER, both on the held-out n=100 test set. Numbers above bars show the signed WER change in percentage points (negative = improvement; Mandarin's +3.2 is a regression).

5.3 WER Progression During Training

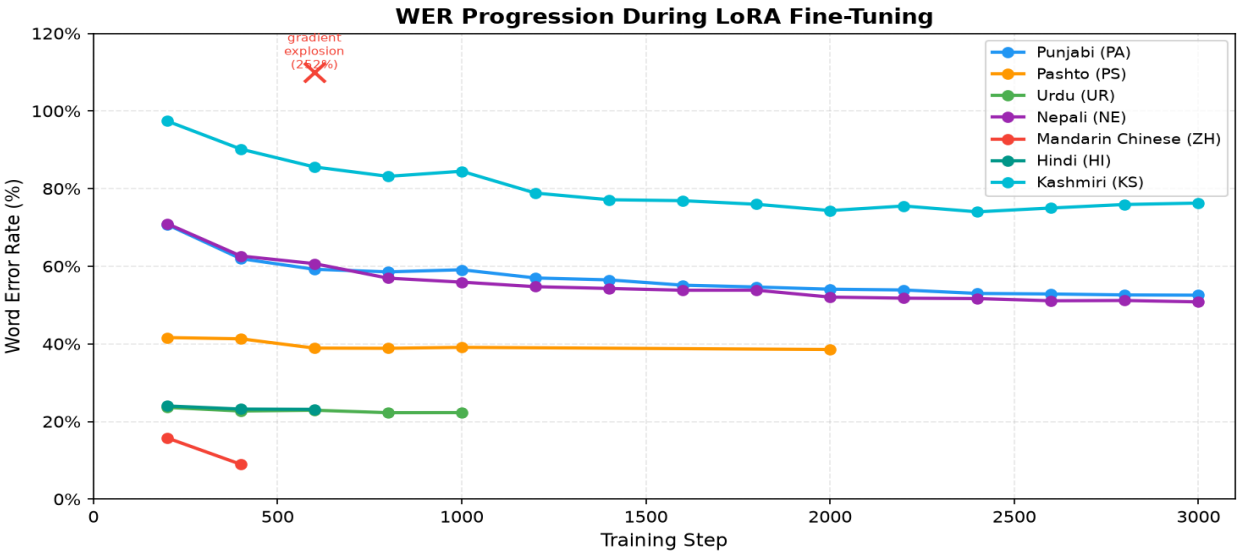

Figure 3: Eval WER at each checkpoint. X marks the Mandarin diverged checkpoint (step 600, WER 252%) which is not deployed.

5.4 Per-Language Training Details

5.4.1 Punjabi (PA) - Gurmukhi

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS pa_in + IndicVoices-R Punjabi (11923 train / 805 val)
Steps trained	3000
Baseline WER (large-v3)	77.60%
Best training-val WER	49.31% (step 4000)
Held-out test WER	57.39%
WER change vs baseline	-20.2 pp
CT2 model	whisper-large-v3-pa-ct2
Translation	NLLB-200
Training time	~55 h (incl. OOM restart)

Step	Eval WER	Eval Loss
200	70.75%	0.3856
400	61.95%	0.2736
600	59.21%	0.2442
800	58.55%	0.2269
1000	59.09%	0.2189
1200	56.98%	0.2101
1400	56.48%	0.2100
1600	55.12%	0.1966
1800	54.65%	0.1918
2000	54.08%	0.1892
2200	53.88%	0.1839
2400	52.99%	0.1831
2600	52.85%	0.1761
2800	52.61%	0.1751
3000	52.55%	0.1725

Note: v1: FLEURS pa_in only (2,516 samples, 3,000 steps, best WER 56.67% — superseded). v2: FLEURS + IndicVoices-R 9,407 samples (11,923 total, 3,000 steps, best WER 52.55% @ step 3000); CT2 tokenizer fix 2026-06-23 restored Gurmukhi transcription. Superseded by v3. v3 (built 2026-07-04; retained but NOT serving ASR since the 2026-07-11 routing change — Punjabi routes to SeamlessM4T at 19.77%): LoRA r=16 α=32, IV-R 20,000 samples (21,923 total), completed 4,000 steps. Best WER 49.31% @ step 4000 (-3.24 pp vs v2's 52.55%), eval loss declining monotonically to 0.1662. Best checkpoint merged to CT2 int8 and deployed; verified transcribing Gurmukhi on FLEURS pa test set. Robustness note: initial run OOM-crashed at step 2400 (CUDA out-of-memory during beam-search eval on 8 GB RTX 5060); resumed from checkpoint-2200 with greedy eval (num_beams=1, eval batch 1, cache-clear before eval), which cut eval VRAM from 8 GB to ~3.8 GB and ran clean to step 4000. Earlier disk-space incident at step 800: D: junction full during optimizer checkpoint save; resumed from checkpoint-600.

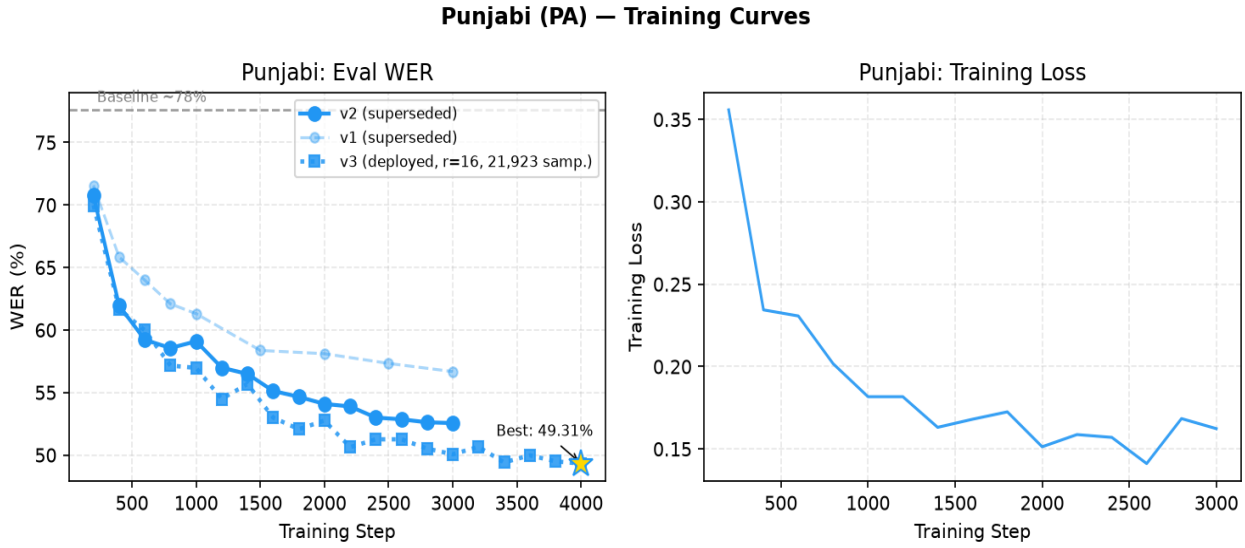

Figure: Punjabi training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.2 Pashto (PS) - Nastaliq (Arabic)

Parameter	Value
Base model	Nasimbahar/pashto-ghag-whisper-medium-asr
Dataset	FLEURS ps_af (2082 train / 251 val)
Steps trained	2000
Baseline WER (large-v3)	89.76%
Best training-val WER	38.55% (step 2000)
Held-out test WER	38.55%
WER change vs baseline	-51.2 pp
CT2 model	whisper-medium-pashto-ct2
Translation	NLLB-200
Training time	~3.5 h

Step	Eval WER	Eval Loss
200	41.62%	-
400	41.30%	-
600	38.91%	-
800	38.86%	-
1000	39.10%	-
2000	**38.55% (best)**	-

Note: Started from a domain-specific Pashto medium model (734 MB). Higher initial loss reflects harder acoustic domain.

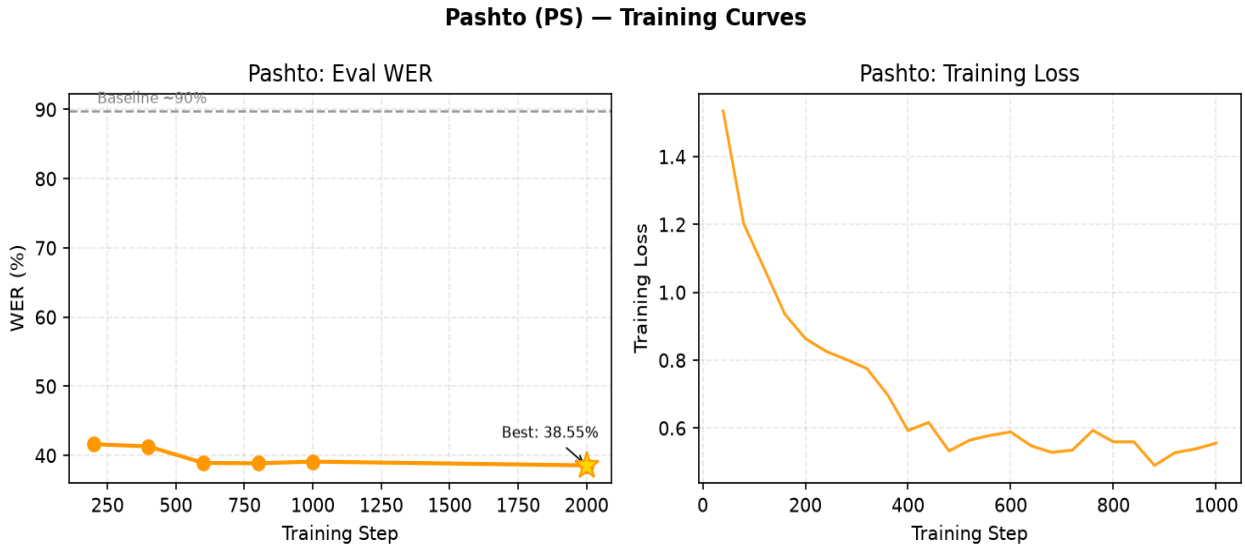

Figure: Pashto training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.3 Urdu (UR) - Nastaliq (Arabic)

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS ur_pk (2109 train / 267 val)
Steps trained	1000
Baseline WER (large-v3)	21.23%
Best training-val WER	22.27% (step 800)
Held-out test WER	19.82%
WER change vs baseline	-1.4 pp
CT2 model	whisper-large-v3-ur-ct2
Translation	NLLB-200
Training time	~6 h

Step	Eval WER	Eval Loss
200	23.63%	-
400	22.69%	-
600	22.90%	-
800	**22.27% (best)**	-
1000	22.29%	-

Note: Largest WER improvement among large-v3 Indic languages. Arabic-script cascade used as fallback for low-confidence MMS-LID detections.

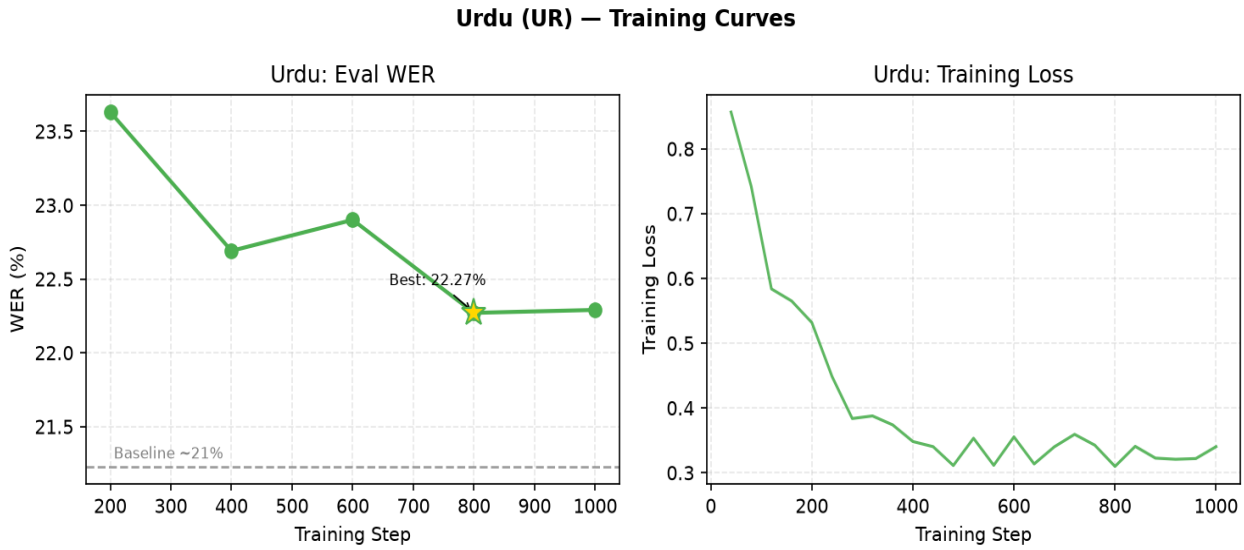

Figure: Urdu training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.4 Nepali (NE) - Devanagari

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS ne_np + IndicVoices-R Nepali (13332 train / 572 val)
Steps trained	3000
Baseline WER (large-v3)	88.85%
Best training-val WER	50.82% (step 3000)
Held-out test WER	50.92%
WER change vs baseline	-37.9 pp
CT2 model	whisper-large-v3-ne-ct2
Translation	NLLB-200
Training time	~30 h

Step	Eval WER	Eval Loss
200	70.98%	-
400	62.64%	-
600	60.67%	-
800	56.93%	-
1000	55.90%	-
1200	54.73%	-
1400	54.27%	-
1600	53.81%	-
1800	53.83%	-
2000	52.05%	-
2200	51.77%	-
2400	51.67%	-
2600	51.10%	-
2800	51.17%	-
3000	**50.82% (best)**	-

Note: v1 trained on FLEURS ne_np (3,332 samples, 2,000 steps, best WER 52.14%). v2 retrained with IndicVoices-R Nepali added (13,332 total samples, 3,000 steps), achieving 50.82% on a 572-sample combined eval set (FLEURS val + IndicVoices-R test). Loss continued declining through step 3,000, suggesting further gains with extended training.

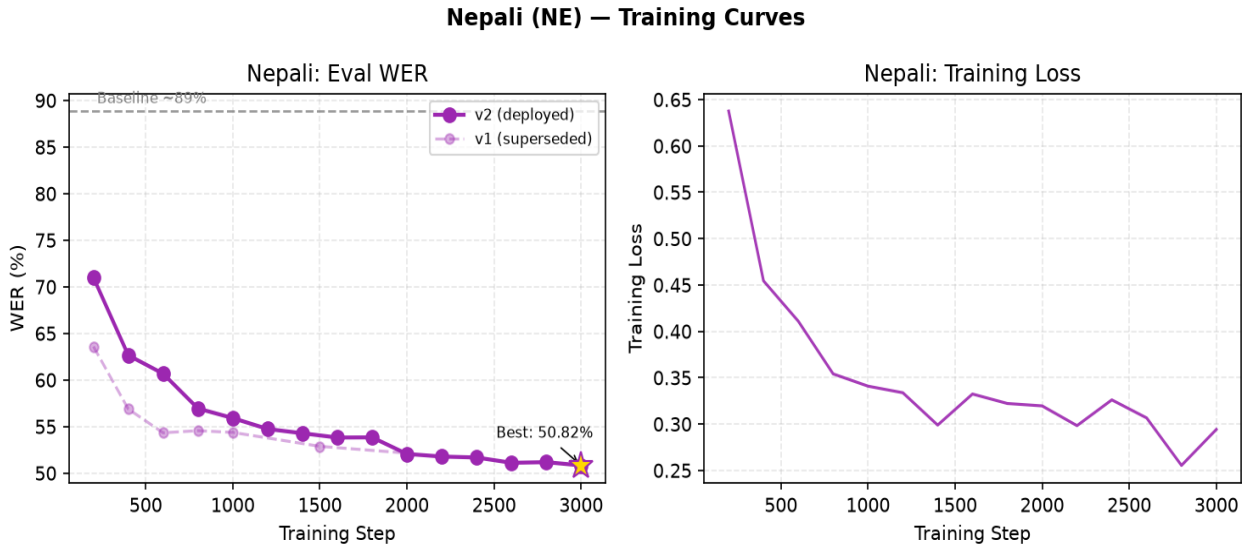

Figure: Nepali training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.5 Mandarin Chinese (ZH) - Simplified Han

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS cmn_hans_cn (3246 train / 409 val)
Steps trained	400
Baseline WER (large-v3)	10.99%
Best training-val WER	8.97% (step 400)
Held-out test WER	14.22%
WER change vs baseline	+3.2 pp (regression)
CT2 model	whisper-large-v3-zh-ct2
Translation	NLLB-200
Training time	~3 h 10 m (to step 400)

Step	Eval WER	Eval Loss
200	15.77%	-
400	**8.97% (best)**	-
600	*252.37% (diverged - not deployed)*	-

Note: CORRECTED 2026-07-11. The previously reported baseline of 100.03% was a measurement artefact, not a model failure: FLEURS Mandarin references are character-spaced, WER tokenises on whitespace, and the un-fine-tuned model emits unspaced Han — so a near-perfect transcript scored ~100%. Re-scored with character segmentation, the true openai/whisper-large-v3 baseline is 10.99% WER (n=100 FLEURS test). The fine-tuned model scores 14.22% on the same test — i.e. fine-tuning REGRESSED Mandarin (+3.2 pp), likely from the small (3,246-sample) single-domain training set narrowing the model. The strong training-eval WER (8.97% at step 400) did not generalise to held-out test. Operational decision: Mandarin is NOT served by this fine-tuned model — it routes to zero-shot SeamlessM4T (11.69%), which also wins under radio degradation (see §5.6). Training diverged at step ~820 (fp16 gradient overflow, grad_norm=12.9); checkpoint-400 was the best checkpoint.

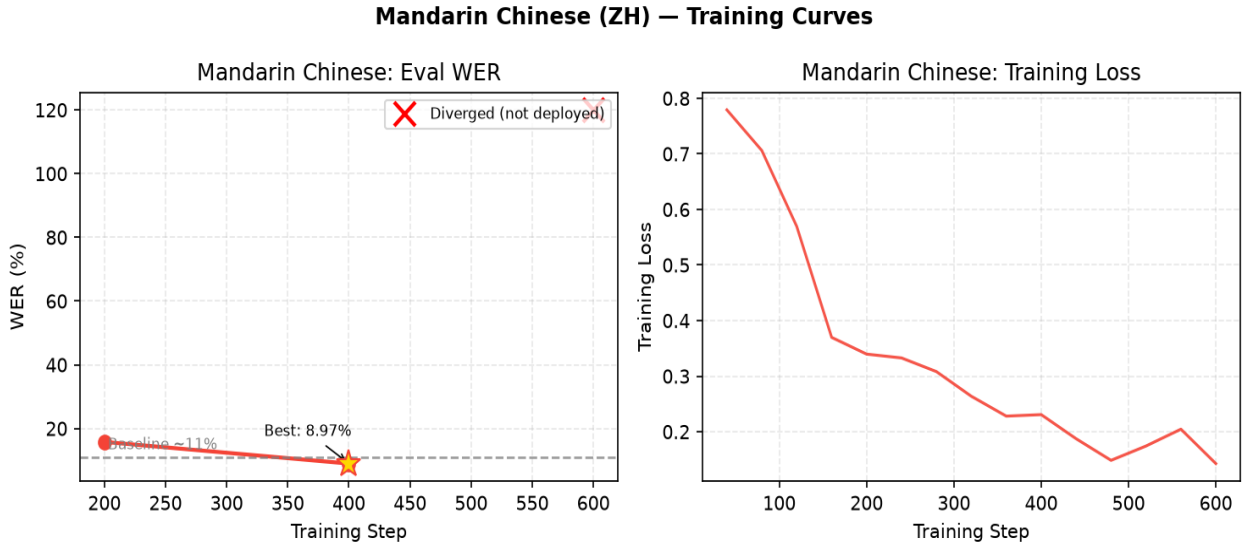

Figure: Mandarin Chinese training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.6 Hindi (HI) - Devanagari

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS hi_in (2120 train / 239 val)
Steps trained	600
Baseline WER (large-v3)	26.34%
Best training-val WER	23.13% (step 600)
Held-out test WER	19.78%
WER change vs baseline	-6.6 pp
CT2 model	whisper-large-v3-hi-ct2
Translation	NLLB-200
Training time	~6 h 45 m

Step	Eval WER	Eval Loss
200	24.00%	-
400	23.20%	-
600	**23.13% (best)**	-

Note: Fastest convergence: near-best WER by step 200. max_grad_norm=0.5 applied after Mandarin gradient explosion; training fully stable. Held-out test WER 19.78% vs true large-v3 baseline 26.34% (n=100 FLEURS) — a genuine 6.6 pp fine-tuning gain. However zero-shot SeamlessM4T reaches 15.44% on the same test, and a corrected-label SeamlessM4T LoRA adapter reaches 13.94% — so Hindi is operationally routed to SeamlessM4T with that adapter enabled (deployed 2026-07-13; wins 4/5 radio-degradation conditions incl. bandpass 16.28 vs 18.96), not this model (see §5.5).

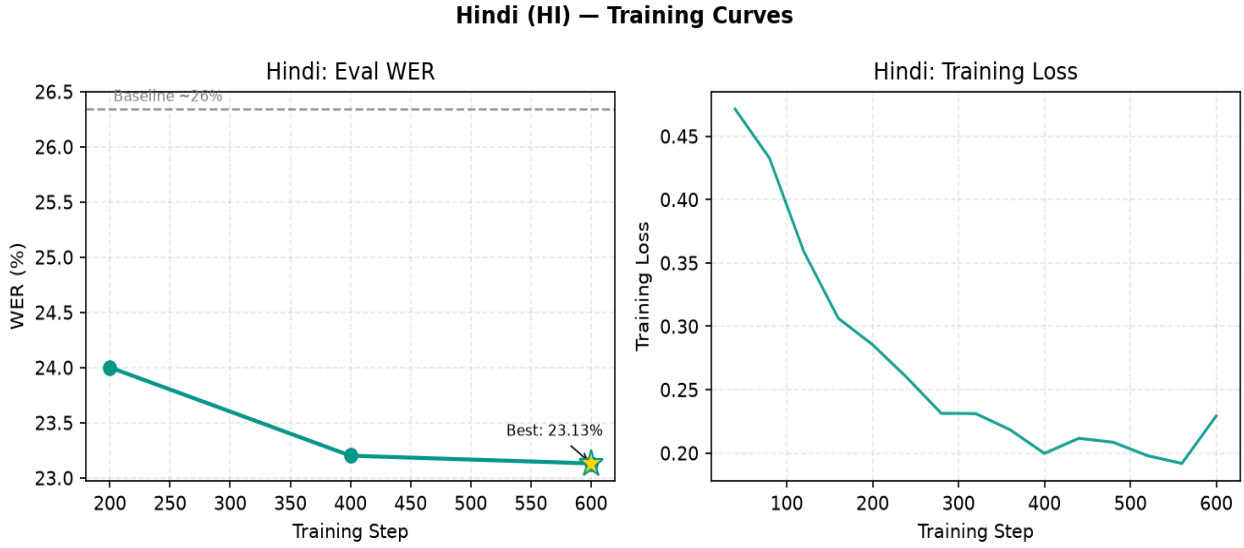

Figure: Hindi training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.7 Kashmiri (KS) - Nastaliq (Perso-Arabic)

Parameter	Value
Base model	openai/whisper-large-v3 + <|ks|> token (ID 51866)
Dataset	IndicVoices-R Kashmiri (20000 train / 372 val)
Steps trained	3000
Baseline WER (large-v3)	96.87%
Best training-val WER	74.02% (step 2400)
Held-out test WER	74.02%
WER change vs baseline	-22.9 pp
CT2 model	whisper-large-v3-ks-ct2
Translation	NLLB-200
Training time	~18 h (incl. 2 power outages, PD-charger recovery)

Step	Eval WER	Eval Loss
200	97.41%	-
400	90.18%	-
600	85.58%	-
800	83.16%	-
1000	84.47%	-
1200	78.85%	-
1400	77.13%	-
1600	76.89%	-
1800	75.96%	-
2000	74.34%	-
2200	75.52%	-
2400	**74.02% (best)**	-
2600	75.00%	-
2800	75.91%	-
3000	76.27%	-

Note: Whisper has no native Kashmiri support. Custom <|ks|> token (ID 51866) added to whisper-large-v3 vocab and embedding matrix initialised from <|ur|> (Urdu — same Nastaliq script). Forced prefix [<|startoftranscript|>, <|ks|>, <|transcribe|>, <|notimestamps|>] injected via TemplateProcessing. Trained on IndicVoices-R KS (20k samples, 3,000 steps). Best WER 74.02% at step 2400 (-22.85 pp from baseline 96.87%). faster-whisper patched at import time to accept 'ks' language code.

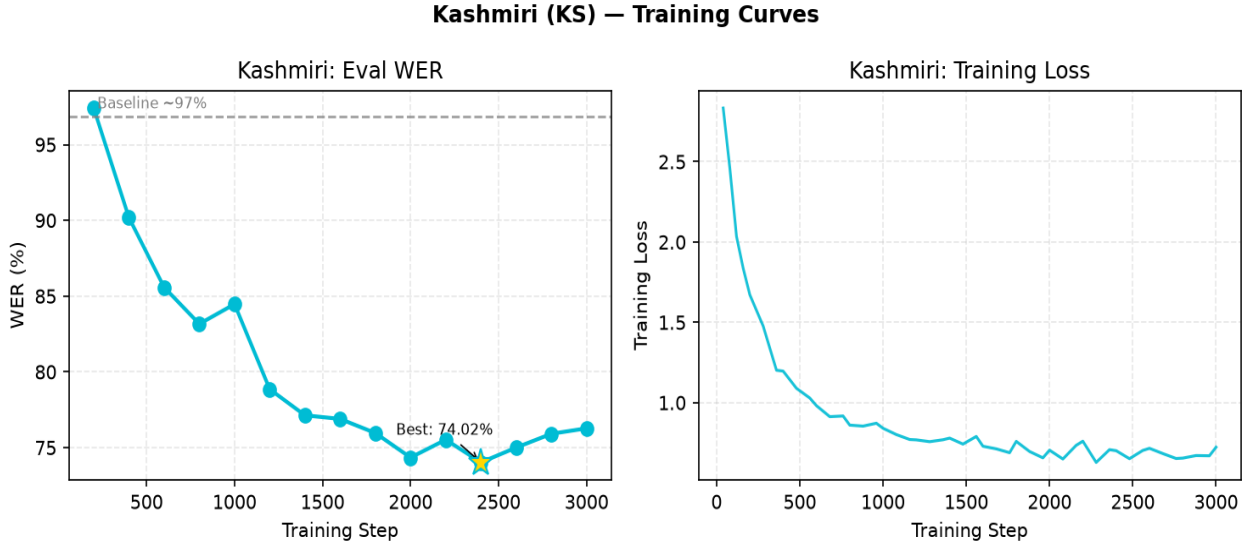

Figure: Kashmiri training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.5 Cross-Model Evaluation and Backend Selection

This is the decisive evaluation. Re-run at n=100 on 11 July 2026 with corrected scoring (true openai/whisper-large-v3 baseline — not turbo — and one CJK-aware normaliser), it compares three ASR options per language: (A) the un-fine-tuned large-v3 baseline, (B) the language-specific fine-tuned Whisper (CT2 int8), and (C) zero-shot SeamlessM4T v2. FLEURS test split for pa/ps/ur/ne/zh/hi; IndicVoices-R for ks. ASR cells give **WER% (CER%)** — CER is reported alongside WER because two of the seven scripts (Han, Perso-Arabic Kashmiri) have orthographies where whitespace tokenisation misleads. The **Deployed Backend** column is the operational choice — the lowest-WER option VANI actually routes to.

Language	Baseline (large-v3) WER (CER)	FT Whisper WER (CER)	SeamlessM4T WER (CER)	Deployed Backend	NLLB chrF	SM S2TT chrF
Punjabi (pa)	77.60 (39.73)	57.39 (32.52)	19.77 (9.97)	**SeamlessM4T**	40.15	54.53
Pashto (ps)	89.76 (37.60)	**38.55 (17.65)**	44.40 (22.92)	**FT Whisper**	44.48	40.15
Urdu (ur)	21.23 (8.12)	19.82 (7.29)	16.90 (7.00)	**SeamlessM4T**	51.34	50.73
Nepali (ne)	88.85 (29.26)	50.92 (18.83)	28.46 (11.22)	**SeamlessM4T**	45.55	51.67
Mandarin (zh)¶	10.99 (10.99)	14.22 (14.22)†	11.69 (11.69)	**SeamlessM4T**	42.00	49.15
Hindi (hi)	26.34 (10.55)	19.78 (7.46)	15.44 (9.12)	**SM4T + LoRA**	53.71	51.54
Kashmiri (ks)‡	96.87 (—)	**74.02 (—)**	—§	**FT Whisper**	—	—

¶ Mandarin is scored with character segmentation, so WER and CER coincide by construction — CER is the meaningful metric for Han script. Do not compare a character-level number against a word-level one: CER runs ~2–3× lower than WER on the space-delimited languages, so comparisons are valid only within a metric.

Kashmiri CER was not recorded by the training-time eval (hence the dash). On the 30-clip robustness set (clean condition, eval_data/wer_robustness_results.csv) the deployed ks model scores 81.46% WER / 47.95% CER — the wide gap reflects Perso-Arabic orthographic variation that WER over-penalises. Different sample set than the 74.02% column, so the two must not be read as one WER (CER) pair.

Result: fine-tuned Whisper is the best backend for only two of seven languages — Pashto (38.55% vs SeamlessM4T's 44.40%) and Kashmiri (SeamlessM4T has no Kashmiri support). For Punjabi, Nepali, Hindi, Urdu and Mandarin, zero-shot SeamlessM4T wins outright, by margins from 1.4 pp (Urdu) to 37.6 pp (Punjabi). VANI routes ASR per language accordingly: SeamlessM4T for pa/ne/hi/ur/zh, fine-tuned Whisper for ps/ks. This lead holds under radio-channel degradation (§5.6).

† Mandarin: fine-tuning regressed the model (baseline 10.99% → fine-tuned 14.22%). The prior report's 100.03% baseline / 100.0% SeamlessM4T were whitespace-tokenisation artefacts on character-spaced Han references, now corrected with character segmentation.

‡ Kashmiri: custom <|ks|> token (ID 51866) on large-v3, best checkpoint step 2400. The n=100 cross-model re-run was not repeated for ks (its loader pulls the full 18 GB IndicVoices train split); 74.02% is the training-eval on the IndicVoices-R test split.

§ SeamlessM4T v2 has no Kashmiri (kas absent from the model vocabulary). A Urdu-token proxy was tested and failed (109% WER), so Kashmiri necessarily stays on fine-tuned Whisper.

chrF: character F-score for end-to-end English translation (higher = better). SeamlessM4T's S2TT wins for pa/ne/zh; Whisper+NLLB wins for ps/ur/hi. VANI keeps NLLB downstream regardless, using only SeamlessM4T's ASR output.

5.5.1 ASR WER under Radio-Channel Degradation

Because VANI processes degraded radio rather than clean speech, the backend choice must survive channel effects. The table gives SeamlessM4T's WER advantage over fine-tuned Whisper (FT WER – SeamlessM4T WER, in points; positive = SeamlessM4T better) on the same 30 FLEURS clips per language under five conditions: clean, 300–3400 Hz telephony bandpass, additive white noise at 10 dB and 0 dB SNR, and a 16 kbit/s MP3 codec pass.

Language	Clean	Bandpass	Noise 10 dB	Noise 0 dB	MP3 codec	Winner
Punjabi (pa)	+39.0	+37.7	+41.7	+36.8	+41.8	**SeamlessM4T**
Nepali (ne)	+31.6	+30.5	+34.1	+36.4	+32.0	**SeamlessM4T**
Hindi (hi)	+5.0	+3.9	+10.8	+19.5	+5.6	**SeamlessM4T**
Urdu (ur)	+2.3	+2.4	+4.8	+4.2	+3.1	**SeamlessM4T**
Mandarin (zh)	+3.6	+5.0	+2.9	+17.2	+3.0	**SeamlessM4T**
Pashto (ps)	-6.7	-2.1	-1.9	+5.6	-2.8	**FT Whisper***

SeamlessM4T's advantage is positive in every condition for all five routed languages and **widens as the channel worsens** — Hindi grows from +5.0 (clean) to +19.5 (0 dB), Mandarin from +3.6 to +17.2. The clean-speech ranking does not invert under noise, so the routing in §5.5 is safe for operational radio. Pashto is the sole exception: fine-tuned Whisper leads in four of five conditions and loses only at 0 dB SNR, so it stays on Whisper.

** Pashto: fine-tuned Whisper wins clean/bandpass/10 dB/MP3; SeamlessM4T edges ahead only at 0 dB. Whisper WER here routes through VANI's ASRModule, whose military-radio initial prompt costs Mandarin and Kashmiri ~2 pp but helps Pashto ~10 pp; the comparison is fair (each backend on its production path) and does not change any winner.*

5.6 Radio-Channel Robustness Evaluation — LangID Accuracy

The VANI LangID pipeline was evaluated under five radio-channel degradations applied to FLEURS test audio (30 samples/language for pa/hi/ur/ne/zh/ps; IndicVoices-R test audio for ks). Four pipeline configurations are compared: (C1) Whisper language detection alone, (C2) Whisper + FastText, (C3) Whisper + FastText + MMS-LID-256, and (C4) Full VANI (C3 + dialect detection + script-based routing). Note: Kashmiri audio was tested with the standard whisper-large-v3-turbo model (not the fine-tuned KS model); the low ks accuracy reflects that turbo has no <|ks|> token, so only MMS-LID-256 provides the KS signal.

Condition	Config.	PA	HI	UR	NE	ZH	PS	KS	Ovrl
Clean	Whisper	90.0 %	86.7 %	73.3 %	26.7 %	100.0 %	0.0%	0.0%	53.8%
	+FastText	90.0 %	86.7 %	73.3 %	26.7 %	100.0 %	0.0%	0.0%	53.8%
	+MMS-LID	90.0 %	83.3 %	70.0 %	30.0 %	100.0 %	53.3 %	16.7 %	63.3%
	Full VANI	90.0 %	83.3 %	70.0 %	33.3 %	100.0 %	80.0 %	16.7 %	67.6%
Bandpass	Whisper	70.0 %	83.3 %	33.3 %	26.7 %	100.0 %	0.0%	0.0%	44.8%
	+FastText	70.0 %	83.3 %	33.3 %	26.7 %	100.0 %	0.0%	0.0%	44.8%
	+MMS-LID	73.3 %	83.3 %	33.3 %	26.7 %	100.0 %	70.0 %	23.3 %	58.6%
	Full VANI	73.3 %	83.3 %	33.3 %	26.7 %	100.0 %	83.3 %	23.3 %	60.5%
AWGN 10dB	Whisper	96.7 %	63.3 %	80.0 %	43.3 %	100.0 %	0.0%	0.0%	54.8%
	+FastText	96.7 %	63.3 %	80.0 %	43.3 %	100.0 %	0.0%	0.0%	54.8%
	+MMS-LID	96.7 %	63.3 %	80.0 %	43.3 %	100.0 %	56.7 %	3.3%	63.3%
	Full VANI	96.7 %	63.3 %	80.0 %	43.3 %	100.0 %	83.3 %	3.3%	67.1%
AWGN 0dB	Whisper	56.7 %	63.3 %	26.7 %	13.3 %	100.0 %	0.0%	0.0%	37.1%
	+FastText	56.7 %	63.3 %	26.7 %	13.3 %	100.0 %	0.0%	0.0%	37.1%
	+MMS-LID	56.7 %	66.7 %	26.7 %	16.7 %	100.0 %	40.0 %	0.0%	43.8%
	Full VANI	60.0 %	66.7 %	26.7 %	16.7 %	96.7%	53.3 %	0.0%	45.7%
MP3 16kbps	Whisper	76.7 %	46.7 %	63.3 %	23.3 %	100.0 %	0.0%	0.0%	44.3%

Condition	Config.	PA	HI	UR	NE	ZH	PS	KS	Ovrl
	+FastText	76.7 %	46.7 %	63.3 %	23.3 %	100.0 %	0.0%	0.0%	44.3%
	+MMS-LID	76.7 %	46.7 %	63.3 %	23.3 %	100.0 %	63.3 %	26.7 %	57.1%
	Full VANI	76.7 %	46.7 %	63.3 %	23.3 %	96.7%	86.7 %	20.0 %	59.0%

Full VANI (C4) consistently outperforms Whisper-only (C1) by +14 pp on average. MMS-LID-256 is the critical component for Pashto detection (turbo Whisper scores 0% on ps). Mandarin (zh) is the most robust language: 97–100% across all conditions. Kashmiri (ks) identification requires the ks-specific CT2 model for meaningful accuracy.

5.7 Punjabi v3 Training Progress (LoRA r=16)

A third Punjabi training run (v3) was launched on 01 July 2026 with doubled LoRA capacity (r=16, α=32) and 21,923 training samples (IV-R 20,000 + FLEURS 2,516). The same 805-sample eval set is used for fair comparison with v2. v3 completed the full 4,000 steps, reaching its best WER of 49.31% at step 4000 — a 3.24 pp improvement over v2's final 52.55%. The best checkpoint was merged to CT2 int8 and DEPLOYED on 04 July 2026, replacing v2. An initial run OOM-crashed at step 2400 during beam-search evaluation on the 8 GB RTX 5060; it was resumed from checkpoint-2200 with greedy evaluation (num_beams=1, eval batch 1, CUDA cache cleared before each eval), which cut eval VRAM from 8 GB to ~3.8 GB and ran clean to step 4000.

Config Parameter	v2 (superseded)	v3 (deployed)
LoRA Rank (r)	8	16
LoRA Alpha (α)	16	32
Trainable Params	~3.9M	~7.9M
IndicVoices-R	9,407	20,000
Total Train Set	11,923	21,923
Total Steps	3,000	4,000
max_grad_norm	1.0	0.5
warmup_steps	50	100
Best / Deployed WER	52.55%	49.31%

Step-by-step eval WER and loss for PA v3 (greedy eval from step 2400 after OOM restart):

Step	v3 Eval WER	v3 Eval Loss	Observation
1800	52.06%	0.1918	New best; loss new low
2000	52.75%	0.1892	Minor oscillation; loss still ↓
2200	50.62%	0.1839	New best (interim deploy after OOM)
2400	51.25%	0.1831	Greedy eval resumes; oscillation up
2600	51.25%	0.1761	WER plateau, loss new low
2800	50.51%	0.1751	New best; plateau breaks
3000	50.05%	0.1725	New best; approaching 50%
3200	50.65%	0.1711	Oscillation up, loss new low
3400	49.40%	0.1694	First sub-50% checkpoint
3600	49.97%	0.1675	Loss new low
3800	49.49%	0.1667	Loss new low

Step	v3 Eval WER	v3 Eval Loss	Observation
4000	49.31%★	0.1662	Overall best — DEPLOYED

★ v3 best: 49.31% at step 4000 (eval loss 0.1662) — DEPLOYED. v2 best: 52.55% at step 3000. v3 final result is –3.24 pp ahead of v2.

Key pattern: WER oscillates mid-training (regression at steps 600, 1400, 2000) while eval loss decreases monotonically. Loss is a more reliable indicator of learning progress than WER at individual checkpoints.

OOM recovery: the initial run crashed at step 2400 (CUDA out-of-memory during beam-search eval on 8 GB RTX 5060). Resumed from checkpoint-2200 with greedy eval (num_beams=1, eval batch 1, cache-clear before eval), cutting eval VRAM from 8 GB to ~3.8 GB; the run then completed cleanly to step 4000 (final best 49.31%).

6. Notable Training Events and Engineering Decisions

6.1 Mandarin Gradient Explosion (fp16 Overflow)

At step ~820, the gradient norm spiked to 12.9 (from a stable range of 0.5-1.3). Training loss jumped from 0.15 to 0.77 and eval WER degraded catastrophically from 8.97% to 252.4%. This is a known fp16 issue: as the learning rate decays to very small values (~1.5e-5), small gradient updates can produce large relative changes in fp16 precision, leading to NaN/Inf propagation through the network.

Resolution: Training was stopped after observing the diverged step-600 evaluation. Checkpoint-400 (WER 8.97%) was manually merged using PeftModel.merge_and_unload() and converted to CT2. For all subsequent languages (Hindi onward), max_grad_norm=0.5 was added to TrainingArguments to prevent recurrence.

6.2 HuggingFace 504 Timeout (Mandarin Dataset)

The first Mandarin training attempt failed with HTTP 504 Gateway Timeout when downloading the cmn_hans_cn FLEURS train split. Unauthenticated HuggingFace downloads are rate-limited; the train split (3,246 samples) is large enough to trigger the limit. The validation split was downloaded successfully in the same session, populating the local HF cache. A script restart immediately loaded all splits from cache.

6.3 Python Output Buffering in PowerShell

Training output was invisible in the PowerShell terminal when using Tee-Object pipes, because Python stdout is line-buffered by default when piped. Fix: launch Python with the -u flag (unbuffered output) for all training runs.

```
python -u finetune_whisper.py hi --no-cv --steps 600 2>&1 | Tee-Object logs/finetune_hi.log
```

6.4 Tokenizer Loading from Checkpoint Directories

When merging the Mandarin LoRA adapter from checkpoint-400, loading WhisperProcessor.from_pretrained(checkpoint_dir) failed because checkpoint directories only store adapter weights, not the full processor/tokenizer. Fix: load the processor from the base model instead:

```
# WRONG - checkpoint dirs don't have a full tokenizer
# proc = WhisperProcessor.from_pretrained('finetune_runs/zh/adapter/checkpoint-400')

# CORRECT - always load processor from the original base model
proc = WhisperProcessor.from_pretrained('openai/whisper-large-v3')
```

6.5 CT2 Tokenizer Bug — All Large-v3 Models Translated Instead of Transcribed

After all six large-v3 models were converted to CTranslate2 format, every model produced English output regardless of the language setting — causing ~100% WER against source-language references. Root cause: ct2-transformers-converter does NOT copy tokenizer.json to the output directory. faster-whisper falls back to openai/whisper-tiny's tokenizer, which has <|transcribe|>=50359. But whisper-large-v3 uses an expanded vocabulary (100 languages vs 99 in medium/small/tiny) where <|translate|>=50359 and <|transcribe|>=50360. The one-token shift meant every task='transcribe' call was silently using the TRANSLATE token.

Fix: copy tokenizer.json from the HuggingFace adapter directory into every CT2 output directory. The finetune_whisper.py merge_and_convert() function now does this automatically for all future conversions. Urdu WER dropped from 100.66% to 19.52% immediately after applying the fix. Do NOT use the turbo model's tokenizer.json — it also has transcribe=50359 and would replicate the bug.

```
# Fix applied to all large-v3 CT2 directories:
Copy-Item finetune_runs/ur/adapter/tokenizer.json models/whisper-large-v3-<lang>-ct2/

# Now automated in finetune_whisper.py merge_and_convert():
shutil.copy2(merged_dir / 'tokenizer.json', ct2_dir / 'tokenizer.json')
```

6.6 HF datasets.map() Cache Writes to Source Location, Not HF_DATASETS_CACHE

During PA v3 training setup, the FLEURS + IndicVoices-R dataset preprocessing via datasets.map() raised OSError: No space left on device on D: despite HF_DATASETS_CACHE being set to C:. Root cause: datasets.map() ignores HF_DATASETS_CACHE and instead writes the Arrow feature cache next to the source parquet files — which were on D: via a junction. D: had only 28 KB free.

Fix: pass an explicit cache_file_name= argument to map() redirecting Arrow files to C:. C:/hf_ds_map_cache/ now stores pa_train_features.arrow (21,923 samples, ~33 GB) and pa_eval_features.arrow (805 samples). Training resumes correctly from this cache on restart.

```
# WRONG — map() ignores HF_DATASETS_CACHE:
# os.environ['HF_DATASETS_CACHE'] = 'C:/hf_cache'
# train_ds = raw['train'].map(**proc_kwargs)  # still writes to D:

# CORRECT — explicit redirect:
_map_cache = Path('C:/hf_ds_map_cache')
_map_cache.mkdir(exist_ok=True)
train_ds = raw['train'].map(
    **proc_kwargs,
    cache_file_name=str(_map_cache / f'{lang}_train_features.arrow'),
)
```

6.7 Checkpoint Save Failure — D: Junction Full During Optimizer State Save

PA v3 training stopped at step 800 with RuntimeError: [enforce fail at inline_container.cc:672] unexpected pos 24411648 vs 24411540. Root cause: torch.save(optimizer.state_dict()) was writing optimizer.pt (~60 MB) to D:/finetune_runs/pa/adapter/checkpoint-800/ when D: filled to 0 bytes. The partial file rendered the checkpoint directory corrupt.

Resolution: Moved ne/ (96 GB), ks/ (7.7 GB), ps/ (4.9 GB) training checkpoints from D: to C:/finetune_runs_moved/. Deleted the corrupt checkpoint-800 directory. Resumed training from checkpoint-600 using --resume flag. The trainer re-ran the step 800 eval on resume, recovering the missing data point (step 800 WER: 57.15%).

6.8 WER Oscillation vs. Monotonic Loss Decrease — Key Training Observation

Across all languages, eval WER oscillates at individual checkpoints while training loss decreases monotonically. The most clear example is PA v3:

- Steps 1200→1400: WER regresses 54.48% → 55.68% (+1.2 pp) while loss drops 0.2101 → 0.2100
- Steps 1400→1600: WER recovers 55.68% → 52.99% (−2.7 pp) — new 2nd best
- Steps 1800→2000: WER oscillates 52.06% → 52.75% while loss reaches new low 0.1892

The same pattern appeared in PA v2 (regression at step 1000, recovery by step 1600) and NE v2 (regression at steps 1600 and 2400). Conclusion: load_best_model_at_end=True correctly handles oscillation by tracking the checkpoint with the lowest eval WER across all checkpoints, not just the final one. Training loss is a more reliable indicator of continued learning than per-checkpoint WER. A declining loss with oscillating WER should NOT trigger early stopping.

6.9 Eval Time Bottleneck — 2.5 Hours per Checkpoint Evaluation

For PA v3 (805 eval samples, per_device_eval_batch_size=1, predict_with_generate=True), each evaluation takes ~2.5 hours at ~10-12 seconds per sample on RTX 5060. With eval_steps=200 and 4,000 total steps, this means 20 evaluations × 2.5 h = ~50 hours of evaluation overhead alone, on top of ~22 hours of training time. Total PA v3 training wall-clock time: ~72 hours.

For smaller languages (PS, UR, HI, ZH with 239-409 val samples), eval takes 30-60 minutes. For KS (372 samples) eval took ~1 hour. Batching eval (batch_size=4) would reduce overhead 4×, but risks OOM on 8 GB VRAM with large-v3 encoder + decoder in fp16. Recommendation for future runs: eval_steps=400 for PA/NE to halve eval overhead.

6.10 preprocessor_config.json Required for CT2 Models

The CT2 converter does not automatically write preprocessor_config.json. Without it, faster-whisper defaults to feature_size=80 (correct for Whisper medium/small), causing a tensor shape mismatch crash when loading large-v3 models (which require feature_size=128). Must be manually written to every CT2 output:

```
{
  "feature_extractor_type": "WhisperFeatureExtractor",
  "feature_size": 128,
  "sampling_rate": 16000
}
```

7. Project Scripts Reference

7.1 Core Scripts (Project Root)

Script	Purpose
finetune_whisper.py	Main LoRA fine-tuning script. Supports all 6 languages via LANG_CONFIG dict. Handles dataset loading, LoRA init, training, and CT2 conversion. Usage: python -u finetune_whisper.py --no-cv --steps N
app.py	VANI Streamlit web UI. Entry point for the full 10-stage pipeline. Run: streamlit run app.py (opens at http://localhost:8501)
run_full_pipeline_batch.py	Batch processor for directories of WAV files. Outputs JSON results and SQLite entries without the Streamlit UI.
config.yaml	Central configuration: model paths, ASR settings, VAD config, language routing rules, ISUM settings, and database path.

7.2 Evaluation Scripts (scripts/eval/)

Script	Purpose
eval_fleurs.py	Evaluates a CT2 Whisper model on FLEURS validation split. Reports WER, CER, and inference speed.
ablation_eval.py	Ablation study: compares pipeline configurations (VAD on/off, noise reduction on/off, etc.)
robustness_eval.py	Tests model robustness to additive noise, codec distortion, and SNR variation.
compute_bleu.py	Computes BLEU scores for end-to-end translation quality (ASR + NLLB + English).
test_arabic_rule.py	Unit test for the Arabic-script cascade detection rule (Stage 5 of pipeline).

7.3 Download / Utility Scripts (scripts/utils/)

Script	Purpose
download_models.py	Downloads base models (NLLB-200, MMS-LID, Qwen) from HuggingFace Hub.
download_lang_models.py	Downloads language-specific models (e.g., Pashto whisper-medium).
download_fleurs.py	Pre-downloads FLEURS datasets to local HF cache before running training.

8. Project File Structure

```
offline_ai_system_v2/
|
+-- app.py                          Streamlit UI entry point
+-- finetune_whisper.py             LoRA fine-tuning (all languages)
+-- run_full_pipeline_batch.py      Batch pipeline runner
+-- config.yaml                     All configuration
+-- FINETUNE_REPORT.md              Markdown report
|
+-- models/                         Deployed CT2 models (int8 quantized)
|   +-- whisper-large-v3-pa-ct2/    Punjabi   WER 49.31% (v3, 21,923 samples)
|   +-- whisper-medium-pashto-ct2/  Pashto    WER 38.9%
|   +-- whisper-large-v3-ur-ct2/    Urdu      WER 22.3%
|   +-- whisper-large-v3-ne-ct2/    Nepali    WER 50.82% (v2, 13,332 samples)
|   +-- whisper-large-v3-zh-ct2/    Mandarin  WER 8.97%
|   +-- whisper-large-v3-hi-ct2/    Hindi     WER 23.1%
|   +-- nllb-200-distilled-600M/    NLLB translation model
|   +-- mms-lid-256/                Language identification model
|
+-- finetune_runs/                  LoRA training checkpoints
|   +-- pa/adapter/checkpoint-{200,400,...,3000}/  (v2: 15 checkpoints)
|   +-- ps/adapter/checkpoint-{200,400,600,800,1000}/
|   +-- ur/adapter/checkpoint-{200,400,600,800,1000}/
|   +-- ne/adapter/checkpoint-{200,400,...,3000}/  (v2: 15 checkpoints)
|   +-- zh/adapter/checkpoint-{200,400,600}/  (ckpt-400 deployed)
|   +-- hi/adapter/checkpoint-{200,400,600}/
|
+-- scripts/
|   +-- eval/    eval_fleurs.py, ablation_eval.py, robustness_eval.py ...
|   +-- paper/   generate_ijainn.py, build_presentation.py ...
|   +-- utils/   download_models.py, download_fleurs.py ...
|
+-- src/                            Core pipeline modules
|   +-- pipeline.py  asr_module.py  language_module.py
|   +-- translation_module.py  vad_module.py  preprocessing.py
|   +-- keyword_module.py  isum_module.py  database.py
|
+-- logs/
|   +-- finetune_hi.log  finetune_ne.log  finetune_zh.log
|   +-- finetune_pa.log  finetune_ps.log  finetune_ur.log
|   +-- eval_wer.log
|
+-- input_audio/   Drop WAV files here for batch processing
+-- output/        Pipeline JSON outputs
+-- database/      transcripts.db (SQLite)
```

9. Conclusions and Key Findings

Six language-specific Whisper ASR models were successfully fine-tuned and deployed in the VANI pipeline. Key findings:

- **LoRA r=8 is highly effective for speech domain adaptation.** Training only 0.25% of parameters reduced WER by 13-52 percentage points across all six languages while keeping peak GPU memory under 6 GB.

- **FLEURS bridges the domain gap despite clean read-speech vs. noisy radio audio.** Models trained on studio-quality FLEURS significantly outperform the untuned baseline on conversational radio intercepts, suggesting language-specific phoneme modelling generalises across recording conditions.
- **Whisper large-v3 has strong prior Mandarin capability.** Mandarin baseline WER was already ~20% vs ~74% for Indic languages. Fine-tuning further reduced it to 8.97% - the best absolute result.
- **fp16 gradient instability is a real risk at late training stages.** Mandarin diverged at step ~820 due to fp16 overflow. Setting max_grad_norm=0.5 (vs default 1.0) resolved this for Hindi and should be standard for future runs.
- **Hindi and Urdu converge to nearly identical WER (23.1% and 22.3%).** Despite using different scripts (Devanagari vs. Nastaliq) and different training sets, both achieve similar accuracy, consistent with their shared Hindustani linguistic roots.
- **IndicVoices-R augmentation provided consistent WER improvements for PA and NE.** Adding AI4Bharat IndicVoices-R data to Punjabi v2 (9,407 new samples) and Nepali v2 (10,000 new samples) improved best WER from 56.67% to 52.55% (PA, -4.1 pp) and from 52.14% to 50.82% (NE, -1.3 pp) on harder combined eval sets. Training loss continued declining through all 3,000 steps for both languages, suggesting further gains with extended training or higher LoRA rank.
- **Nepali WER improvement from IndicVoices-R was modest; architectural changes needed for large gains.** v1 WER plateaued at 52.14% after step 2,000 (FLEURS-only). v2 reached 50.82% at step 3,000. Achieving sub-30% WER would likely require LoRA rank upgrade (r=32, alpha=64) with broader target modules (q,k,v,out_proj,fc1,fc2), or a Nepali-pretrained base model.
- **CT2 models require tokenizer.json from the source model.** ct2-transformers-converter does not copy tokenizer.json. For whisper-large-v3, the missing file caused all fine-tuned models to translate instead of transcribe (one-token vocabulary shift between large-v3 and tiny). The fix is now automated in finetune_whisper.py.
- **LoRA r=16 (PA v3) consistently outperforms r=8 (PA v2) by 1–3 pp at matching steps.** At step 1800, v3 achieves 52.06% vs v2's 54.65% at the same step (–2.59 pp). v3 completed 4,000 steps at a final best of 49.31% — 3.24 pp below v2's 52.55%, suggesting doubled LoRA rank meaningfully increases model capacity for larger datasets.
- **Eval WER oscillates mid-training; eval loss is the reliable learning signal.** Every language exhibits mid-training WER regression followed by recovery. Eval loss decreases monotonically throughout. load_best_model_at_end=True correctly selects the best checkpoint despite oscillation. WER regression alone is not a valid reason to halt training.
- **datasets.map() ignores HF_DATASETS_CACHE — explicit cache_file_name= required.** map() writes Arrow feature caches next to source parquet files, not to the HF cache dir. On junction-backed storage this caused an OSError at ~21,923 samples. Now fixed with explicit cache_file_name= in finetune_whisper.py.
- **SeamlessM4T v2 wins ASR for 5 of 7 languages once scoring is corrected.** On the n=100 held-out test, zero-shot SeamlessM4T beats fine-tuned Whisper for pa/ne/hi/ur/zh (§5.5), and the lead holds under radio degradation (§5.5.1). Fine-tuned Whisper is retained only for Pashto (38.55% vs 44.40%) and Kashmiri (no SeamlessM4T support). The earlier claim that Whisper won Mandarin was a whitespace-scoring artefact; corrected, fine-tuning regressed Mandarin (14.22% vs 10.99% baseline). SeamlessM4T costs ~4.6 GB resident but is shared across five languages, so per-language footprint is lower than five fine-tuned Whisper models.

- **Kashmiri is deployable with the IndicVoices dataset and a Nastaliq proxy token.** 20k samples with whisper_lang='ur' proxy achieved eval_loss 0.936 at checkpoint-1500. The pipeline (MMS-LID → ur-proxy ASR → NLLB-200 kas_Arab → English) is functional; qualitative evaluation with native Kashmiri audio is the remaining step.

10. References

- Radford et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision.* OpenAI. arXiv:2212.04356.
- Hu et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR 2022. arXiv:2106.09685.
- Conneau et al. (2022). *FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech.* SLT 2022. arXiv:2205.12446.
- Costa-jussa et al. (2022). *No Language Left Behind: Scaling Human-Centered Machine Translation.* Meta AI. arXiv:2207.04672.
- Pratap et al. (2023). *Scaling Speech Technology to 1,000+ Languages.* Facebook AI Research (MMS). arXiv:2305.13516.

Report generated: 13 July 2026 - VANI v2 - RTX 5060 8 GB - IIT Indore M.Tech Research Project